Pakistan Navy Complaint Management System (CMS)
Document 03: Role-Based Access Control (RBAC) Manual

Document Identifier: CMS-RBAC-V1.2.4
Target Audience: System Administrators, Security Auditors, IT Directorate
Date: September 2026
Status: Approved for Handover

---

1. Executive Summary & RBAC Philosophy

The **Role-Based Access Control (RBAC)** subsystem of the Pakistan Navy Complaint Management System (CMS) provides a granular, deterministic, and auditable access governance architecture.

1.1 Key Architecture Shift in Version `v1.2.4`

Prior to release `v1.2.4`, the application utilized legacy hierarchical permission bubbling, where granting a parent module (e.g. `employees` or `complaints`) automatically granted implicit access to child modules (e.g. `designation`, `category`, `sub-category`).

As of `v1.2.4`, the system enforces Strict Exact-Match Permission Isolation:
* Automatic permission inheritance is **completely eliminated**.
* Every module permission is evaluated independently.
* If a role is granted `employees` but NOT `designation`, the user CAN view employee rosters but CANNOT view, create, or alter designations.
* This guarantees the principle of least privilege required in military and secure government environments.

---

2. The 19 Granular System Modules

The platform divides administrative operations into **19 discrete module boundaries**:

#	Module Slug	Human-Readable Name	Description & Protected Scope
1	`dashboard`	Executive Dashboard	High-level KPI widgets, complaint volume graphs, and status summaries.
2	`users`	Administrative Users	Management of back-office staff login accounts (`users` table).
3	`frontend-users`	Portal / Mobile Users	Administration of web/mobile resident user profiles (`frontend_users`).
4	`cmes`	CMES Formations	Commander Military Engineering Services units and commands.

#	Module Slug	Human-Readable Name	Description & Protected Scope
5	`city`	Cities Directory	Geographic cities hosting naval establishments.
6	`sector`	Housing Sectors	Residential housing sectors/divisions linked to CMES units.
7	`roles`	Roles & Permissions	Creation, modification, and permission assignment for system roles.
8	`employees`	Staff & Technicians	Maintenance technicians, staff roster, and field personnel records.
9	`designation`	Job Designations	Trades and ranks (Electrician, Plumber, Mason, Carpenter, etc.).
10	`houses`	Accommodations / Houses	Residential quarters, barracks, and accommodation units.
11	`complaints`	Complaint Management	Core ticket registry, dispatching, state transitions, and resolution.
12	`category`	Complaint Categories	Primary complaint divisions (e.g. Civil, Electrical, Water Supply).
13	`sub-category`	Sub-Categories	Second-tier classifications (Internal Wiring, Sanitary, Windows).
14	`complaint-titles`	Complaint Titles	Specific fault definitions (Ceiling Fan Burnt, Pipe Burst).
15	`approvals`	Approval Performas	Multi-item spare requisitions, Work/Maintenance performas.
16	`spares`	Spares & Inventory	Warehouse stock, item catalogs, vendor details, and stock logs.
17	`reports`	Analytics & Reports	Comprehensive operational summaries, exportable PDFs, and CSVs.
18	`sla`	SLA Rules Engine	Service Level Agreement parameters, escalation thresholds.
19	`registered-devices`	Registered Devices	Hardware IMEI/UUID binding registry for mobile app security.

---

3. Comprehensive Role & Permission Matrix

The following matrix illustrates the standard role allocations recommended for naval operations:

Module Slug	Super Admin (`ID=1`)	CMES / XEN Officer	GE / Supervisor	Helpdesk Operator	Storekeeper (Inventory)	Read-Only Auditor
`dashboard`	✅ Yes	✅ Yes	✅ Yes	✅ Yes	✅ Yes	✅ Yes
`users`	✅ Yes	❌ No	❌ No	❌ No	❌ No	❌ No
`frontend-users`	✅ Yes	✅ Yes	❌ No	✅ Yes	❌ No	❌ No
`cmes`	✅ Yes	👁️ View	❌ No	❌ No	❌ No	👁️ View
`city`	✅ Yes	👁️ View	❌ No	❌ No	❌ No	👁️ View
`sector`	✅ Yes	👁️ View	👁️ View	❌ No	❌ No	👁️ View
`roles`	✅ Yes	❌ No	❌ No	❌ No	❌ No	❌ No
`employees`	✅ Yes	✅ Yes	✅ Yes	👁️ View	❌ No	👁️ View
`designation`	✅ Yes	✅ Yes	👁️ View	❌ No	❌ No	👁️ View
`houses`	✅ Yes	✅ Yes	✅ Yes	✅ Yes	❌ No	👁️ View
`complaints`	✅ Yes	✅ Yes	✅ Yes	✅ Yes	👁️ View	👁️ View
`category`	✅ Yes	✅ Yes	👁️ View	❌ No	❌ No	👁️ View
`sub-category`	✅ Yes	✅ Yes	👁️ View	❌ No	❌ No	👁️ View
`complaint-titles`	✅ Yes	✅ Yes	👁️ View	❌ No	❌ No	👁️ View
`approvals`	✅ Yes	✅ Yes (Approve)	✅ Yes (Create)	❌ No	👁️ View	👁️ View
`spares`	✅ Yes	👁️ View	👁️ View	❌ No	✅ Yes (Full CRUD)	👁️ View
`reports`	✅ Yes	✅ Yes	✅ Yes	👁️ View	👁️ View	✅ Yes
`sla`	✅ Yes	✅ Yes	👁️ View	❌ No	❌ No	👁️ View
`registered-devices`	✅ Yes	❌ No	❌ No	👁️ View	❌ No	👁️ View

Legend: ✅ Full Access (Create/Read/Update/Delete) | 👁️ View Only | ❌ No Access (Menu hidden, HTTP 403 Forbidden on direct URL)

---

4. Technical Implementation & Code Architecture

4.1 Permission Evaluation in Eloquent (`app/Models/Role.php`)

The permission engine uses direct exact-match matching:

```
public function hasPermission(string $permission): bool
{
    // Superadmin bypass: Role ID 1 or role_name 'admin' has immutable full access
    if ($this->id === 1 || $this->role_name === 'admin') {
        return true;
    }

    // Extract module name from "module.action" format (e.g. "users.view" -> "users")
    $module = explode('.', $permission)[0];

    // Retrieve active module permissions for this role
    $permissions = $this->rolePermissions->pluck('module_name')->toArray();

    // Strict exact comparison (no wildcards or parent fallbacks)
    return in_array($module, $permissions, true);
}
```

4.2 Superadmin Auto-Assignment Hook

To ensure system stability during updates, `Role::boot()` guarantees that `role_id = 1` immediately receives all 19 modules whenever a role record is created or updated:

```
static::saved(function ($role) {
    if ($role->id === 1) {
        $modules = [
            'dashboard', 'users', 'frontend-users', 'cmes', 'city', 'sector',
            'roles', 'employees', 'designation', 'houses', 'complaints',
            'category', 'sub-category', 'complaint-titles', 'approvals',
            'spares', 'reports', 'sla', 'registered-devices'
        ];
        // Automatic updateOrCreate sync...
    }
});
```

4.3 Route Protection Middleware (`routes/web.php`)

Every administrative route group is strictly protected by the permission middleware:

```
// CMES Formation routes - Strictly guarded by cmes.view
Route::prefix('cmes')->name('cmes.')->middleware('permission:cmes.view')->group(function () {
    Route::get('/', [CmeController::class, 'index'])->name('index');
    Route::post('/', [CmeController::class, 'store'])->name('store');
    Route::put('/{cme}', [CmeController::class, 'update'])->name('update');
    Route::delete('/{cme}', [CmeController::class, 'destroy'])->name('destroy');
});

// House Management routes - Strictly guarded by houses.view
Route::prefix('houses')->name('houses.')->middleware('permission:houses.view')->group(function () {
    Route::get('/', [HouseController::class, 'index'])->name('index');
    // ...
});
```

4.4 Sidebar UI Synchronization (`resources/views/layouts/sidebar.blade.php`)

To prevent visual confusion or broken links, UI navigation elements strictly match the route permission:

```
@if($user && ($user->hasPermission('houses')))
<li class="nav-item">
    <a href="{{ route('admin.houses.index') }}" class="nav-link">
        <i class="fas fa-home"></i>
        <span>Accommodations</span>
    </a>
</li>
@endif
```

---

5. Administrator Step-by-Step Guide for Role Management

5.1 Creating a New Role

1. Log in to the CMS administrative portal with a Super Administrator account.
2. Navigate to **Administration** → **Roles & Permissions** (`/admin/roles`).
3. Click the **"Create New Role"** button at the top-right corner.
4. Enter the **Role Name** (e.g., `Sector_Supervisor_West`) and an informative **Description**.
5. You will be presented with a list of checkable module cards.
6. Select the specific modules this role is authorized to operate.
7. Click **"Save Role"**.

5.2 Editing Existing Role Permissions

1. In the **Roles** list, locate the target role and click **"Edit"** (`/admin/roles/{id}/edit`).
2. Toggle the module checkboxes according to updated operational duties.
3. Click **"Update Permissions"**. Changes take effect immediately upon the user's next HTTP request.

5.3 Assigning a Role to a User

1. Navigate to **User Management** → **System Users** (`/admin/users`).
2. Click **Edit** on the target user account.
3. In the **Role** dropdown selector, choose the desired role.
4. Click **"Save User"**.

---

6. Post-Deployment Protocol: Sub-Permission Re-Ticking

> [!IMPORTANT]
> **Mandatory Notice for System Administrators:**
> In versions prior to `v1.2.4` , granting `Employees` automatically granted `Designation` , and granting `Complaints` automatically granted `Categories` and `Titles` .
> Because `v1.2.4` decoupled all permissions for maximum security, **administrators must inspect existing custom roles in `/admin/roles`** .
> If an existing role needs access to Designations, Categories, Sub-Categories, or Complaint Titles, the administrator MUST edit that role and explicitly tick those checkboxes. (The Super Administrator `role_id = 1` is updated automatically and requires no manual action).